Below is a **practical, day-to-day DevSecOps "master list"** of the **most commonly used classes/tools + methods/commands** across the full DevSecOps lifecycle (**Plan → Code → Build → Test → Scan → Release → Deploy → Operate → Monitor → Respond**) with **real-time use cases + examples**, in a **table format**.

✅ Note: DevSecOps doesn't mean "Java classes only". In real projects, DevSecOps work is done using **tools + APIs + CLIs + YAML/DSL pipelines**. So here, "classes & methods" means **the core objects/APIs and the most used commands/functions** developers and DevOps/SRE/security engineers use daily.

---

1) Git (SCM) – Day-to-day DevSecOps (Source Control + Security)

Tool / Library	Common Commands ("Methods")	Real-Time Use Case	Example
Git CLI	git clone, git checkout, git pull, git push	Daily code collaboration & branching	git checkout -b feature/auth
Git CLI	git merge, git rebase	Integrate branches cleanly	git rebase main
Git CLI	git revert, git reset, git reflog	Hotfix rollback or recovery	git revert <commitId>
Git CLI	git tag, git describe	Release versioning	git tag v1.2.0
Git CLI	git diff, git blame	Security audit / root cause	git blame src/Auth.java
GPG (Git Signing)	git commit -S	Signed commits for supply-chain security	git commit -S -m "fix"

---

2) GitHub / GitLab / Bitbucket (Repo + PR security)

Platform	Common "Methods"	Use Case	Example
GitHub	PR Review + branch protection	Prevent insecure code merging	Require approvals + status checks
GitHub Actions	workflow_dispatch, secrets, env	CI/CD secure automation	Use secrets.DOCKER_TOKEN
GitLab CI	stages/jobs, variables	Full pipeline automation	.gitlab-ci.yml
CODEOWNERS	Ownership rules	Security team approval required	/infra/* @sec-team

---

3) CI/CD (Jenkins / GitHub Actions / GitLab CI / Azure DevOps)

Tool	Common Classes / Keywords / Commands	Use Case	Example
Jenkins Pipeline	pipeline, stage, steps, environment	CI automation	stage('Build'){ sh 'mvn clean package' }
Jenkins	withCredentials, credentialsId	Secrets handling	withCredentials([string(...)])
GitHub Actions	jobs, steps, uses, run	Automated build/test/scan	uses: actions/setup-java@v4
GitLab CI	stages, script, rules, artifacts	Build & deploy gating	rules: if $CI_COMMIT_BRANCH=="main"
Azure Pipelines	stages/jobs/steps, task, variables	Enterprise CI	task: Maven@3

---

4) Build Tools (Maven / Gradle / Node / Python)

Tool	Common Methods/Commands	Use Case	Example
Maven	clean, compile, test, package, verify, deploy	Java build + publish	mvn clean verify
Maven Plugins	maven-surefire-plugin, failsafe, shade, jacoco	tests/coverage/uber jar	mvn jacoco:report
Gradle	build, test, bootJar, publish	Java/Spring build	./gradlew build
Node/NPM	npm install, npm ci, npm run build, npm audit	Frontend & scanning	npm audit –production
Python pip	pip install, pip freeze	deps management	pip freeze > req.txt

---

5) Secure Code / Quality / Linting (SAST & QA)

Tool	Common Methods	Use Case	Example
SonarQube	Quality gate + rules	Enforce security/quality	Block merge if gate fails
ESLint	eslint, rules configs	JS security rules	npx eslint .
Checkstyle	checkstyle:check	Java style/security policies	mvn checkstyle:check
SpotBugs	spotbugs:check	Java bug+security detection	Find NPE & injection patterns
Semgrep	semgrep scan	SAST quickly	semgrep --config=p/owasp-top-ten

---

6) Secrets & Key Management (Security Core)

Tool	Common Methods / Commands	Use Case	Example
HashiCorp Vault	vault login, vault kv get/put, policies	Secure secrets centrally	vault kv get secret/app
AWS Secrets Manager	GetSecretValue	Fetch DB password securely	App reads at runtime
AWS KMS	Encrypt, Decrypt, key rotation	Encrypt data/secrets	Encrypt .env or tokens
SOPS	sops -e/-d	Encrypt YAML/JSON in Git	sops -e k8s-secret.yaml
git-secrets	scan commits	Prevent key leakage	Block AWS key commits

---

7) Containers (Docker / Podman)

Tool	Common Commands	Use Case	Example
Docker	build, run, push, pull	Containerize app	docker build -t app:1.0 .
Docker	exec, logs, inspect	Troubleshooting	docker logs -f app
Docker Compose	up, down, ps	local microservices	docker compose up –d
Dockerfile	FROM, RUN, COPY, USER	Secure image builds	USER nonroot

---

8) Container Security / Supply Chain Security (Critical DevSecOps)

Tool	Common Methods	Use Case	Example
Trivy	trivy image, trivy fs	container+code scan	trivy image nginx:latest
Grype	grype <image>	vulnerability scan	grype app:1.0
Syft	SBOM generation	dependencies inventory	syft app:1.0 -o spdx-json
Cosign	cosign sign, verify	signed images	Sign image before deploy
SBOM	SPDX/CycloneDX formats	compliance + audits	attach SBOM in release

---

9) Artifact Repositories (Release & Governance)

Tool	Common Methods	Use Case	Example
Nexus	upload/download	central artifact store	store maven jars
Artifactory	build-info, promote	release gating	promote staging to prod
Docker Registry	push/pull	container distribution	docker push repo/app:v1

---

10) Kubernetes / Helm (Deployment & Runtime)

Tool	Common Commands	Use Case	Example
kubectl	apply, get, describe, logs	daily deployment ops	kubectl apply -f deploy.yaml
kubectl	exec, port-forward	debug running pods	kubectl exec -it pod – sh

kubectl	rollout status, rollout undo	rollback release	kubectl rollout undo deploy/app
Helm	install, upgrade, rollback	package deployments	helm upgrade --install app chart/
Kustomize	overlays/patches	env-specific config	prod vs staging overlays

---

11) Kubernetes Security (DevSecOps must-have)

Tool	Methods	Use Case	Example
RBAC	Roles/RoleBindings	least privilege	dev cannot delete prod pods
NetworkPolicy	ingress/egress rules	block lateral movement	only allow app→db
Pod Security	restricted/baseline	block privileged pods	disallow runAsRoot
Kyverno	policies + validate/mutate	enforce standards	block :latest tags
OPA Gatekeeper	constraint templates	policy as code	require labels
Falco	runtime detection	detect suspicious exec	alert on shell spawn

---

12) Infrastructure as Code (Terraform / Ansible)

Tool	Common Methods	Use Case	Example
Terraform	init, plan, apply, destroy	cloud infra automation	terraform plan -out tfplan
Terraform	state list, import	state management	import existing IAM
Terraform Security	tfsec/checkov	IaC scanning	detect open S3 buckets

Ansible	playbooks/roles	server provisioning	install Nginx/Java
Ansible	ansible-playbook	config management	ansible-playbook site.yml

---

13) Cloud DevSecOps (AWS / Azure / GCP)

Cloud	Common API/CLI "Methods"	Use Case	Example
AWS IAM	users/roles/policies	least privilege	role for CI deploy only
AWS EKS	cluster + nodegroups	K8s hosting	deploy microservices
AWS CloudWatch	logs/metrics/alarms	monitoring	alarm CPU > 80%
Azure DevOps	pipelines + boards	enterprise delivery	stage gates
GCP IAM	service accounts	secure automation	deploy with SA

---

14) Monitoring / Logging / Observability

Tool	Common Concepts / Methods	Use Case	Example
Prometheus	scraping + PromQL	metrics monitoring	alert on error rate
Grafana	dashboards + alerts	visualization	HTTP latency dashboard
ELK	ingest/search	central logs	query failed login attempts
Loki	log aggregation	cost-effective logs	link logs to traces
OpenTelemetry	instrumentation	traces/metrics/logs unified	trace API request flow

---

15) Incident Response / Security Ops

Tool	Methods	Use Case	Example
PagerDuty	alert routing	on-call incident response	P1 triggers on-call
ServiceNow	tickets + workflow	compliance change record	change approval
SIEM (Splunk etc.)	correlation rules	detect attacks	brute-force detection
WAF	rules	block attacks	block SQL injection patterns

Real-time DevSecOps Example Flow (End-to-End)

Stage	What happens	Tools Used
Code	developer pushes code	Git
CI Build	compile + unit tests	Maven/Gradle
SAST	detect insecure patterns	Semgrep/SonarQube
Secrets Scan	stop leaked keys	git-secrets/TruffleHog
Container Build	build docker image	Docker
Vulnerability Scan	scan image deps	Trivy/Grype

SBOM + Signing	supply-chain security	Syft + Cosign
Deploy	rollout new version	Kubernetes + Helm
Runtime Security	detect anomalies	Falco
Monitor	alerts + dashboards	Prometheus + Grafana
Incident	page SRE/security	PagerDuty + SIEM

If you want, I can also create a **separate table for each DevSecOps category** (CI/CD / Kubernetes / Cloud / Security scanning / Observability) with **even deeper details**, including:
✅ "when to use vs when not to use",
✅ best practices,
✅ and a **sample secure pipeline YAML** (Jenkinsfile + GitHub Actions + GitLab CI).

Below are **separate deep-dive tables for each DevSecOps category**:

- **CI/CD**
- **Kubernetes**
- **Cloud**
- **Security Scanning (SAST/DAST/SCA/Container/IaC/Secrets)**
- **Observability (Logs/Metrics/Traces/Alerts)**

Each table includes:
✅ commonly used features/commands
✅ real-time use cases + examples
✅ **when to use vs when NOT to use**
✅ best practices

---

1) CI/CD (Jenkins / GitHub Actions / GitLab CI / Azure DevOps)

CI/CD Deep Table

Area	Tool	Common keywords / commands	Real-time use case	When to use ✅	When NOT to use ❌	Best Practices ⭐	Example
Pipelines	Jenkins Pipeline	pipeline {}, stage, steps, agent, post	build/test/release automation	When you need custom enterprise CI	If you only need simple workflows (overkill)	Keep pipelines declarative, split into shared libs	stage('Test'){ sh 'mvn test' }
Pipelines	GitHub Actions	jobs, steps, uses, run	CI for PR branches	GitHub repo native CI	If strict air-gapped infra required	Use pinned versions, restrict permissions	permissions: read-all
Security	Secrets handling	secrets, withCredentials, vault	hide tokens/keys	Always for passwords /keys	Never hardcode in YAML or repo	rotate secrets, least access	AWS_SECRET_ACCESS_KEY in secrets
Reuse	Templates	reusable workflows, pipeline templates	standardization	Many microservices org	If pipeline differs per app heavily	golden pipelines with parameters	reuse CI template
Artifact	Artifact store	artifacts, archiveArtifacts	store jar/image, logs	Always for traceability	Not needed for tiny scripts	keep retention rules	store target/*.jar
Governance	Approvals	manual approvals, gates	prevent unsafe prod release	compliance /regulatory org	if team is small & fast prototyping	approvals only for prod stages	prod gate required

Deploy	CD step	kubectl apply, helm upgrade	release into cluster	automated deploy	don't auto-deploy to prod without checks	separate deploy account	helm upgrade – install

---

CI/CD: "When to use vs not use" (Quick guideline)

✅ Use CI/CD when:

- more than 1 developer
- production deployments exist
- you require security scanning and audit

❌ Don't over-engineer CI/CD when:

- single prototype, no release process
- experiments / learning stage

CI/CD Best Practices ⭑ (Top 12)

1. **Branch protections**: require review + status checks
2. **Immutable artifacts**: build once, deploy same artifact
3. **Use least privilege permissions** for pipeline runner
4. **Never store secrets in repo**
5. Use **commit signing** and enforce verified commits
6. Use **quality gates** (SonarQube / tests / security scan)
7. Run **SAST + dependency scan on PR**
8. Use **SBOM generation** and store it with release
9. Use **staging deployment** before prod

10. Enable **rollback strategy** (Helm rollback / rollout undo)
11. Use **timeouts + retries** (avoid hanging pipelines)
12. Full logs + artifact retention for auditing

---

2) Kubernetes DevSecOps (Deploy + Security + Runtime)

Kubernetes Deep Table

Area	Tool/Resource	Commands / YAML Objects	Real-time use case	When to use ✅	When NOT to use ❌	Best Practices ⭐	Example
Deploy	Deployment	replicas, selector, strategy	run microservice	stateless workloads	not for DB with storage	readiness/liveness probes	strategy: RollingUpdate
Release	Helm	install, upgrade, rollback	versioned releases	multi-env release mgmt	if only 1 YAML file and no versions	lock chart versions	helm rollback app 2
Debug	kubectl	logs, exec, describe	diagnose crashes	always in ops	avoid direct exec in prod repeatedly	use ephemeral containers	kubectl logs -f pod
Security	RBAC	Roles/ClusterRole	limit access	always	never use cluster-admin for apps	least privilege roles	app can read configmap only
Security	NetworkPolicy	ingress/egress rules	stop lateral movement	production clusters	if CNI doesn't support it	default deny model	allow app → db only
Security	Secrets	Secret, SealedSecrets	store sensitive env	always	not safe to store raw secret in Git	use sealed secrets + vault	sealed secret

Governance	Admission Control	OPA/Kyverno	enforce policies	secure org	if no policy ownership	policy-as-code	deny privileged pod
Runtime	Pod Security	restricted/ baseline	prevent root pods	production	not for special workloads needing privileges	use restricted everywhere	deny hostNetwork

---

Kubernetes Best Practices ⭑ (Must Follow)

1. **Always run as non-root**
2. Use **resource limits** (cpu/memory)
3. Use **readOnlyRootFilesystem**
4. Use **NetworkPolicies**
5. Use **RBAC least privilege**
6. Enable **audit logs**
7. No :latest image in production
8. Use **HPA** for scaling
9. Use **PDB** (Pod disruption budget)
10. Use separate namespaces (dev/stage/prod)

---

3) Cloud DevSecOps (AWS / Azure / GCP)

Cloud Deep Table

Area	**Tool**	**Methods/ Commands**	**Real-time use case**	**When to use ✅**	**When NOT to use ❌**	**Best Practices ⭑**	**Example**

IAM	AWS IAM	Roles/Policies	secure automation	always	never use root account	role-based access, MFA	CI role only deploy
Secrets	Secrets Manager	GetSecretValue	runtime secret fetch	prod apps	avoid in code env file	rotate, audit access	fetch DB password
Encryption	AWS KMS	Encrypt/Decrypt	encrypt data at rest	compliance	never store plain sensitive data	key rotation	encrypt S3 objects
Network	VPC / SG	inbound/outbound rules	secure networking	always	avoid open 0.0.0.0/0 for DB	least network exposure	db only accessible by app SG
Logging	CloudTrail	API audit logs	track activity	always	never disable	centralize trails	alert on IAM change
Monitoring	CloudWatch	alarms/metrics	infra monitoring	always	don't rely only on CPU metrics	add app metrics	alarm error rate
IaC	Terraform	plan/apply	infra automation	multi env infra	don't click in console in prod	policy-as-code	terraform plan

---

Cloud Best Practices ★

1. **Use IaC only** (Terraform)
2. **No long-lived IAM keys** → prefer roles
3. Enable **CloudTrail** and protect logs
4. Use **private subnets** for DB/internal services
5. Use **WAF** for public endpoints

6. Use **least privilege** IAM policies
7. Enable **encryption at rest + in transit**
8. Centralized logging + SIEM integration

---

4) Security Scanning (DevSecOps Core)

This covers:

- SAST
- DAST
- SCA (dependencies)
- Container scanning
- IaC scanning
- Secrets scanning
- SBOM + Signing

Security Scanning Deep Table

Scan Type	Tool	Common commands	What it finds	When to use ✅	When NOT to use ❌	Best Practices ⭐	Example
SAST	SonarQube	Quality Gate	insecure code patterns	PR checks	don't run only once in prod	enforce gate on merge	block PR if failed
SAST	Semgrep	semgrep scan	injections, secrets patterns	quick CI scan	not full replacement for manual review	use OWASP rules	--config p/owasp-top-ten
SCA	OWASP Dependency-Check	dependency-check	vulnerable libraries	Java projects	avoid ignoring CVEs	fail build for HIGH severity	detect Log4j

SCA	npm audit	npm audit fix	JS dependency CVEs	frontend builds	don't auto-fix without testing	lockfile discipline	npm ci
Secrets	TruffleHog	scan history	leaked keys/tokens	every PR	don't scan only main branch	scan git history too	detect AWS key
IaC	Checkov / tfsec	checkov -d	open ports/storage	terraform pipelines	don't deploy unscanned infra	fail on critical	open S3 bucket detected
Container	Trivy	trivy image	vuln packages	CI build	don't skip base image scan	scan base + runtime	scan app:1.0
SBOM	Syft	syft <image>	inventory list	releases	not optional for regulated systems	attach SBOM	SPDX JSON
Signing	Cosign	cosign sign/verify	authenticity	prod deployments	don't deploy unsigned images	verify in admission policy	deny unsigned image

---

Security Scanning Best Practices ★

1. Run scans **shift-left** (PR stage)
2. Only allow merge if **Quality Gate passes**
3. Use **severity thresholds** (fail on HIGH/CRITICAL)
4. Generate **SBOM** every release
5. Sign images with **Cosign**
6. Run scans on both:
    - **source code**

- **built artifact**
- **container image**

7. Use **policy enforcement** (OPA/Kyverno) to block unsafe deploys

---

5) Observability (Logging + Monitoring + Tracing + Alerting)

Observability Deep Table

Area	**Tool**	**Common methods/ commands**	**Real-time use case**	**When to use ✅**	**When NOT to use ❌**	**Best Practices ★**	**Example**
Metrics	Prometheus	scrape + PromQL	monitor API latency	microservices	don't monitor only infra	RED metrics	rate(http_requests_total[5m])
Dashboards	Grafana	dashboards + alerts	visualize errors	all env	don't create 1000 dashboards	standardized dashboards	API latency board
Logs	ELK / OpenSearch	ingest/query	debugging	prod troubleshooting	don't store sensitive logs	mask PII	"login failed" search
Logs	Loki	log aggregation	lower cost logs	K8s env	not if you need advanced SIEM correlation	label-based logs	query pod logs
Tracing	Jaeger	distributed trace	trace request flow	microservices	if monolith only, less value	trace sampling	trace 500 error

OpenTelemetry	SDK/Collector	instrumentation	unified telemetry	modern apps	don't instrument everything blindly	selective spans	trace DB calls
Alerts	Alertmanager	routing/grouping	incident alert	prod	don't alert on noise	SLO-based alerts	alert error budget burn

---

Observability Best Practices ★

1. Adopt **SLIs/SLOs** (not random alerts)
2. Avoid alert noise: only alert on symptoms
3. Use:
    - **RED** metrics (Request rate, Error rate, Duration)
    - **USE** metrics (Utilization, Saturation, Errors)
4. Correlate metrics ↔ logs ↔ traces
5. Ensure PII masking in logs
6. Retention policy and cost control
7. Centralized dashboards with team ownership

If you want…

I can also provide **production-ready sample CI/CD pipelines** for:
✅ Jenkinsfile
✅ GitHub Actions YAML
✅ GitLab CI YAML

Including security steps:

- SAST + SCA
- Secrets scanning

- IaC scanning
- Container scan
- SBOM + Image signing
- Kubernetes policy enforcement (Kyverno / OPA)